Code as Policies の蒸留
— Gemini を教師にして、小型モデルにロボット制御コードを書かせる —

PAI 最終課題 説明資料　／　omnicampus アカウント名: Hirano15

GitHub	https://github.com/Rinnnnnnnnnnnnnnnnnnnn/PAI_final
動画（YouTube）	https://youtu.be/kXr9rSQjUgc（2 倍速再生）
実行環境	Google Colab（データ作成: CPU ランタイム／学習・評価・動画: GPU ランタイム A100）

1. 課題の概要

Code as Policies（CaP, Liang et al., 2022）は、大規模言語モデル（LLM）に few-shot 例を見せて、自然言語の指示から **ロボットを動かす Python コード**を書かせる手法です。生成したコードをその場で実行すると、ロボットが動きます。 本課題では、公式の卓上マニピュレーションデモ（PyBullet 上の UR5e アームと 2 指グリッパーで、ブロックとボウルを動かす）を題材にして、次の 2 段階で取り組みました。

1. **CaP を Gemini API で動かす**：元の実装で使われていた OpenAI `text-davinci-003` は提供が終わっているため、Gemini で動くように書き直しました。
2. **CaP の蒸留**：Gemini 版 CaP を「教師」にして教師データを自動で作り、手元の GPU で動く小型のコード生成モデル（生徒：**Qwen2.5-Coder-Instruct**）を LoRA でファインチューニングしました。生徒は **few-shot 例なしの短い入力だけ**でロボット制御コードを書きます。

2. 目的

CaP は few-shot プロンプトが数千トークンあり、指示のたびに外部 API へ長いプロンプトを送ります。そのため、料金・レート制限・応答の遅さが問題になります。そこで、次の 3 点を調べることにしました。

- 教師（Gemini＋few-shot）の振る舞いを、**小型モデルの重み**に覚えさせられるか
- 生徒は、教師と比べてどのくらいの**成功率**と**速さ**でタスクをこなせるか
- 学習したタスクの外の指示に、生徒はどこまで**汎化**するか

3. システム構成

CaP は役割の異なる複数の LMP（Language Model Program ＝ LLM ＋ プロンプト ＋ 実行環境）が、お互いを関数として呼び合う構成になっています。

```
指示 "put the red block in the bowl closest to it"
  │
  ▼
tabletop_ui (LMP) ── 指示 → ロボット操作コード（say / put_first_on_second など）
  ├─► parse_obj_name      "一番近いボウル" → 'blue bowl'
  ├─► parse_position      "〜の左 10cm" → 座標 [x, y]
  ├─► parse_question      "ブロックは何個？" → 答え
  ├─► transform_shape_pts 図形の拡大・回転
  └─► fgen（LMPFGen）      コード中の未定義関数（get_closest_idx など）の本体を生成
                                  │
                                  ▼
                PyBullet：UR5e + Robotiq 2F-85 で Pick & Place
```

蒸留では、LMP が「誰にコードを書かせるか」を差し替えられるようにしました（バックエンド）。教師バックエンドは few-shot プロンプト全体を Gemini に渡し、生徒バックエンドは `# LMP: 名前` ・物体リスト・指示コメントだけからなる短い入力を、学習済みのモデルに渡します。LMP 側は、どちらのバックエンドが使われているかを区別しません。

4. 手法

4.1 Gemini 版 CaP（教師）

davinci は「続きを書く」補完型でしたが、Gemini はチャット型なので、そのまま使うとコードブロックの記号・説明文・`import` 文などが出力に混ざります。 そこで、**システム指示**で「コード補完エンジンとして、最後のコメントに対するコードだけを書く」ように指定し、さらに**後処理**（コードブロックの抽出・指示の繰り返しの除去・`import` 行の除去・stop トークンでの打ち切り）を加えて、元の CaP と同じ形の出力が得られるようにしました。

4.2 教師データの自動生成

指示文と、その指示が達成できたかを判定する関数を、シーンに合わせて自動で作る **9 種類のテンプレート**を実装しました。

テンプレート	指示の例	成功判定
block_on_bowl	put the red block on the blue bowl	ブロックがボウルの上にある
color_alias	put the sun-colored block on the blue bowl	色の言い換えを正しく解釈できたか
corner	move the red block to the top left corner	ブロックが角の付近にある
closest_bowl	put the red block in the bowl closest to it	最も近いボウルの上にある
extreme_block	put the left most block on the green bowl	一番端のブロックが置かれた
side_of	put the red block to the left of the blue bowl	ボウルの左／右 4cm 以上、上下のずれ 8cm 以内
stack	stack all the blocks on the green bowl	全ブロックがボウルの上に積まれた
matching	put the blocks in the bowls with matching colors	同じ色のペアがすべて揃った
count	how many blocks are there?	ロボットの発話に正しい数が含まれる

seed からシーン（ブロック 2〜4 個・ボウル 1〜4 個）とタスクを作り、Gemini 版 CaP をシミュレーターで実行しました（150 エピソード）。**成功したエピソードのコードだけ**を教師データにしています。 1 エピソードから `tabletop_ui` ・ `parse_obj_name` ・ `fgen` などの LMP 呼び出しが複数取れるので、それぞれを 1 サンプルにしました。

4.3 生徒の学習（LoRA による SFT）

項目	設定
生徒モデル	Qwen2.5-Coder-7B-Instruct（A100、bf16）。T4 なら 1.5B を自動で選ぶ
LoRA	r = 16, α = 32, dropout = 0.05、対象は全線形層（q/k/v/o/gate/up/down_proj）
学習	3 エポック、学習率 2e-4（cosine）、実効バッチサイズ 16、最大長 2048 トークン、gradient checkpointing
損失	system・user 部分のラベルは −100 にして除外し、assistant（コード＋終了トークン）だけを学習
評価	学習に使っていない seed（100000 番台）30 エピソードで、全テンプレートの成功率と生成時間を測定

5. 工夫した点

1. **チャット型 LLM を補完型として使う**：システム指示と後処理の 2 段構えで、davinci 用の few-shot プロンプトを書き換えずに Gemini で動かしました。この後処理は生徒の出力にもそのまま使っています。
2. **成功判定つきのタスク自動生成**：テンプレートごとに判定関数を用意したので、人がラベルを付けなくても「成功したコードだけ」を集められます。紛らわしい配置（最も近いボウルとの距離差が 3cm 未満など）は、判定がぶれないように生成の段階で除外しました。
3. **長時間の自動実行に耐える安全装置**
   - 生成コードが届かない座標を指定すると、元の環境は無限ループします。シミュレーションのステップ数に上限を設けて打ち切るようにしました（`SafePickPlaceEnv`）。
   - 物体が多いと、元の配置処理は置き場所を探し続けて止まります。同じ乱数で配置を事前に試し、詰まる組み合わせは選び直すようにしました。
   - Gemini 無料枠のレート制限に合わせてリクエスト間隔を空け、429 / 5xx エラーは待ち時間を延ばしながら再試行します。1 日の上限に達したら安全に止まり、翌日に続きから再開できます（結果を JSONL に 1 件ずつ追記）。
4. **公平な比較とデータ分割**：同じ seed なら同じシーンになるので、教師と生徒を同じ条件で比較できます。検証データはエピソード単位で分け（seed が 10 の倍数）、同じシーンのサンプルが学習と検証の両方に入らないようにしました。`fgen` のように毎回同じ入力になる呼び出しは重複を除きました。
5. **短い入力への置き換え**：生徒は few-shot 例を読まないので、教師と比べて入力が大幅に短くなります。教師では数千トークンあった入力が、数十〜数百トークンになります。
6. **CPU / GPU ランタイムの分離**：API を待つだけのデータ作成は CPU ランタイムで、学習は GPU ランタイムで行い、結果は Google Drive で受け渡すようにして、GPU の利用時間を節約しました。
7. **提出動画の自動作成**：学習済みのアダプタを読み込んでタスクを順に実行し、指示・生成コード・成否判定を映像に重ねて 1 本の動画にまとめるノートブック（`CaP_Student_Demo_Video.ipynb`）を作りました。

6. 結果

6.1 教師データ

教師（gemini-3.5-flash-lite）で 150 エピソードを実行し、**128 エピソード（85.3%）で成功**しました。 成功したエピソードの LMP 呼び出しを重複なしで集め、**学習 218 件・検証 24 件**の教師データを作りました。

テンプレート	エピソード数	教師の成功率	教師の主な失敗の原因
block_on_bowl	13	100%	
color_alias	17	100%	
corner	21	100%	
extreme_block	23	100%	
side_of	23	100%	
count	14	100%	
matching	11	55%	存在しない物体名を指定した（環境内の UnboundLocalError）
stack	20	45%	生成した関数の引数名の食い違い（TypeError）、積み方の誤り
closest_bowl	8	25%	関数生成（fgen）の失敗：未定義変数・`get_closest_idx` の登録漏れ

学習データの内訳（学習用）: tabletop_ui 116 件、parse_obj_name 64 件、parse_position 19 件、parse_question 14 件、fgen 5 件。 学習時の損失は、保存先の設定の都合で記録が残っていませんでした。

6.2 生徒の評価（学習に使っていない seed で 30 エピソード）

	成功率	1 エピソードのコード生成時間（平均）※	入力
教師（Gemini + few-shot）	85.3%（150 エピソード）	12.0 秒	few-shot プロンプト全体（数千トークン）
生徒（Qwen2.5-Coder-7B + LoRA）	**86.7%**（26 / 30）	**6.4 秒**	LMP 名・物体リスト・指示のみ

※ 教師の生成時間には、レート制限に合わせた待ち時間が含まれます。教師の値は教師データ作成時（seed 0〜149）のもので、生徒とは seed が異なります。評価 30 エピソードはテンプレートをランダムに選んだため、テンプレートごとの数に偏りがあります（color_alias 9、side_of 5、stack 5、他は 1〜3）。

テンプレート	評価数	生徒の成功率	生徒の失敗の内容
block_on_bowl / corner / side_of / count / extreme_block / closest_bowl	3 / 3 / 5 / 2 / 1 / 1	すべて 100%	
color_alias	9	89%	ocean-colored（青）を別の色のブロックと解釈した（エラーなしで別の物体を動かした）
stack	5	60%	存在しない物体名を指定した（UnboundLocalError ×2）
matching	1	0%	`get_obj_names('blocks')` と存在しない引数で呼んだ（TypeError）

6.3 提出動画でのデモ（生徒：Qwen2.5-Coder-7B-Instruct + LoRA）

学習に使っていない seed（100000 番台）で、テンプレートごとに 1 タスクずつと、学習テンプレートに無い自由な指示 2 つを実行しました（動画は 1 分 23 秒、2 倍速）。 **11 タスクのうち 9 タスクで成功**しました（テンプレート 9 タスクは自動判定で 8 成功、学習外の 2 タスクは目視判定で 1 成功）。

#	テンプレート	指示	結果	生成時間
1	block_on_bowl	put the cyan block on the green bowl	○ 成功	3.9 秒
2	color_alias	put the sun-colored block on the brown bowl	○ 成功（sun-colored → yellow と解釈）	2.0 秒
3	corner	move the orange block to the top right corner	○ 成功	4.9 秒
4	closest_bowl	put the yellow block in the bowl closest to it	○ 成功	13.0 秒
5	extreme_block	put the left most block on the green bowl	○ 成功	11.5 秒
6	side_of	put the gray block to the right of the red bowl	○ 成功	8.0 秒
7	matching	put each block in the bowl of the same color	× 失敗（TypeError）	7.4 秒
8	stack	stack all the blocks on the green bowl	○ 成功	10.5 秒
9	count	how many blocks are there?	○ 成功（"there are 4 blocks"）	7.0 秒
10	学習外	put the red block in the bowl farthest from it	○ 成功（目視判定：一番遠い黄色のボウルに入れた）	12.1 秒
11	学習外	put the blocks in a horizontal line near the bottom side	× 失敗（目視判定：横に並べずに、積み上げてしまった）	8.7 秒

生成時間は、1 タスクの中で行われた全 LMP 呼び出し（下位 LMP・関数生成を含む）のコード生成時間の合計です。

7. 考察と今後の課題

- **few-shot 例なしで、教師と同等の成功率**：生徒は few-shot 例を読まずに、評価 30 エピソードで 86.7% 成功しました。教師データ作成時の教師の成功率は 85.3% で、ほぼ同じ水準です。1 エピソードあたりのコード生成時間は、教師の 12.0 秒に対して生徒は 6.4 秒でした。ただし教師の値には、無料枠のレート制限に合わせた待ち時間（1 リクエスト最大 7.5 秒）も含まれているので、モデルそのものの速度を比べたものではありません。生徒の利点は、そうした API の制限や料金を気にせず、自分の GPU だけで動かせることにあります。ただし seed とテンプレートの分布が教師と異なり、評価数も少ないので、厳密な比較には同じ seed で教師も評価する必要があります（ノートブックの `RUN_TEACHER_EVAL`）。
- **教師が苦手なタスクは、生徒も苦手**：教師の成功率が低かった stack（45%）と matching（55%）では、生徒も失敗しています。成功したエピソードしか学習しないので、これらのテンプレートは学習データが少なくなります（matching は 6 エピソード分）。closest_bowl は教師が 25% しか成功しなかったので学習データが 2 エピソード分しかありませんでしたが、生徒は評価で成功しました（1 エピソードのみ）。
- **few-shot 例なしでも CaP の構造を再現できた**：生徒は、指示に応じて `parse_obj_name`・`parse_position`・`parse_question` などの下位 LMP を呼び分け、`get_obj_positions_np` のような未定義関数を `fgen` に作らせていました。教師の階層的なコード生成のやり方が、プロンプトではなく重みとして身についたといえます。色の言い換え（sun-colored → yellow）のような言語的な推論もできていました。
- **失敗の主な原因は API の誤用**：matching では、生徒が存在しない引数をつけて `get_obj_names('blocks')` と呼んだため、TypeError で止まりました。教師は few-shot 例で API の正しい使い方をその都度見られますが、生徒は短い入力しか見ないため、API の仕様を間違えて覚えると誤りがそのまま出てしまいます。対策としては、利用できる API の一覧（シグネチャ）を生徒の入力に短く入れることや、このパターンの学習データを増やすことが考えられます。
- **汎化の限界**：学習外の「一番遠いボウル」では、closest_bowl のコード（距離でソートして先頭を選ぶ）を末尾 `[-1]` に変えた、妥当なコードを書けました。一方で「横一列に並べる」では、stack テンプレートの書き方をそのまま当てはめて積み上げてしまいました。**学習したパターンを少し変えるのは得意だが、新しい種類の動作は苦手**という、蒸留の特徴がそのまま出た結果です。
- **データの多様化**：Gemini に指示文の言い換えを作らせたり、温度を上げて同じタスクを複数回解かせたりすれば、学習データの多様性を増やせます。
- **失敗データの活用**：今回は捨てた失敗エピソードを「悪い例」として、DPO などの選好学習に使えます。

8. 提出物の構成

ファイル	内容
Code_as_Policies_Interactive_Demo.ipynb	公式デモ（参照用・OpenAI 版）
Code_as_Policies_Gemini_JP.ipynb	Gemini API 版 CaP（日本語解説つき）
CaP_Distillation_Gemini_Teacher_JP.ipynb	教師データ作成・LoRA 学習・評価・結果のまとめ
CaP_Student_Demo_Video.ipynb	学習済み生徒モデルによる実行と提出動画の作成

参考文献

[1] J. Liang et al., “Code as Policies: Language Model Programs for Embodied Control,” ICRA 2023. https://code-as-policies.github.io/

[2] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” ICLR 2022.
[3] Qwen Team, “Qwen2.5-Coder Technical Report,” 2024.